Architecture Learning - Lesson 02

29 October 2026 | Java contracts, ownership, immutability and allocation tradeoffs

Learning objective

Design a registry that preserves configuration invariants after construction, explain where ownership crosses API boundaries, and evaluate defensive copying against allocation cost.

Case: immutable routing registry

A reusable routing library stores ordered rules containing path, HTTP method and priority. Requirements: callers cannot change initialized configuration; diagnostics observes stable values; unnecessary allocations should be avoided.

```
public final class RouteRegistry {
    private final List<RouteRule> rules;
    public RouteRegistry(List<RouteRule> rules) {
        this.rules = Collections.unmodifiableList(rules);
    }
    public List<RouteRule> getRules() { return rules; }
}
```

This implementation exposes an unmodifiable view of a caller-owned list. Although clients cannot modify the list through getRules(), the original owner can modify the backing list.

```
List<RouteRule> input = new ArrayList<>();
input.add(new RouteRule("/users", "GET", 1));
RouteRegistry registry = new RouteRegistry(input);
input.clear();
System.out.println(registry.getRules().size()); // 0
```

Reference identity and final

Java passes object references by value. A final field prevents reassignment of the field after initialization, but does not freeze the referenced object. For references, == tests identity; equals tests logical equality according to the class implementation.

Snapshot versus view

Collections.unmodifiableList(source) is a read-only view backed by source. List.copyOf(source) returns an unmodifiable list containing the source element references at copy time. It may reuse a suitable existing unmodifiable list. The Java API does not promise identical object identity across all implementations. List.of(a, b) is convenient when elements are supplied individually; List.copyOf(existingList) accepts an existing collection. Both reject null elements.

```
List<String> first = List.of("A", "B");
List<String> second = List.copyOf(first);
System.out.println(first == second); // implementation-dependent; not guaranteed
```

Shallow immutability

List.copyOf isolates the collection structure, not mutable elements. Two independent lists can still reference the same StringBuilder.

```
List<StringBuilder> original = new ArrayList<>();
original.add(new StringBuilder("Hello"));
List<StringBuilder> copy = List.copyOf(original);
original.get(0).append(" World");
System.out.println(copy.get(0)); // Hello World
```

Immutable elements and a safe registry

A RouteRule containing only private final String path, String method and int priority, with no mutators and no unsafe construction escape, can be immutable. Constructor parameters initialize state; accepting constructor arguments does not imply subsequent mutability. The registry can copy the list once and return it directly.

```
public final class RouteRule {
    private final String path;
    private final String method;
    private final int priority;
    public RouteRule(String path, String method, int priority) {
        this.path = Objects.requireNonNull(path);
        this.method = Objects.requireNonNull(method);
        this.priority = priority;
    }
    public String getPath() { return path; }
    public String getMethod() { return method; }
    public int getPriority() { return priority; }
}
public final class RouteRegistry {
    private final List<RouteRule> rules;
    public RouteRegistry(List<RouteRule> rules) {
        this.rules = List.copyOf(rules);
    }
    public List<RouteRule> getRules() { return rules; }
}
```

Mutable element: copy at both boundaries

If a field must be a StringBuilder, an incoming reference can be mutated by the caller and a returned reference can be mutated by the receiver. Copy at construction and on return. StringBuilder does not provide a public clone() method.

```
public final class MutableMethodRule {
    private final StringBuilder method;
    public MutableMethodRule(StringBuilder method) {
        this.method = new StringBuilder(
            Objects.requireNonNull(method));
    }
    public StringBuilder getMethod() {
        return new StringBuilder(method);
    }
}
StringBuilder input = new StringBuilder("GET");
MutableMethodRule rule = new MutableMethodRule(input);
input.append("POST");
rule.getMethod().append("DELETE");
System.out.println(rule.getMethod()); // GET
```

Allocation and throughput

Suppose diagnostics reads the method of 100,000 rules once per second. A getter that creates a defensive StringBuilder may create up to 100,000 builders per second, along with backing character storage, before JIT optimization. Allocation pressure can increase garbage collection work. A String getter returns the existing immutable reference without copying. For a stable HTTP method, String is the better representation.

Copying a list of 100,000 references normally costs O(n) time and O(n) reference storage, but does not clone the RouteRule instances. At 1,000 registry creations per second, repeated full snapshots can be expensive. Reusing a suitable immutable input may avoid copying, but identity reuse is not a contractual guarantee. Measure actual throughput, allocation rate and GC behavior using representative workloads rather than assuming a precise cost.

When mutability is useful

StringBuilder is suitable as a local accumulator for incremental concatenation. Convert to String at a stable API boundary. Low access frequency reduces the expense of defensive copying but is not, by itself, a reason to expose or retain mutable state.

```
StringBuilder builder = new StringBuilder();
for (String segment : segments) {
    builder.append(segment);
}
String result = builder.toString();
```

Design boundaries and caveats

Immutability is not synonymous with thread safety of an entire system: publication, surrounding mutable state and compound operations still matter. A read-only view is not a snapshot. Defensive copying should be performed only where ownership requires it; avoid repeated copying of immutable values. Mutable nested fields require their own ownership analysis. Validate input invariants, including null policy and domain constraints, independently of copying.

Practice exercises - solutions intentionally omitted

1. Replace RouteRule.method with a mutable List of allowed methods. Identify the minimum safe copies at constructor and getter boundaries, assuming elements are Strings. 2. Predict the effect of changing an element inside a List after List.copyOf. 3. Design a microbenchmark comparing direct String getters with defensive StringBuilder getters, specifying how to prevent dead-code elimination and which allocation metrics to inspect. 4. Explain when a caller-owned immutable list can safely be shared instead of copied.

Verification and references

Examples were compiled and run on Java 21 with java.util imports, separate source files for public classes, and a small harness. Checks covered the backing-list failure, safe registry snapshots, null rejection, shared mutable elements, both defensive-copy boundaries, and local accumulation. Identity reuse is an observation, not a portable requirement. Exercises remain unsolved and untested; allocation and throughput estimates are illustrative, not benchmark results.

Java 21 List API

Java 21 StringBuilder API

Written by Alby Raju with AI assistance.